



برنامه نویسی شی گرا

Object Oriented Programming

مدرس:

مریم خراشادیزاده

توابع دوست

- **توابع دوست** توابعی هستند که خارج از قلمرو کلاس هستند ولی می‌توانند به اعضای خصوصی یک کلاس دسترسی داشته باشند.
- می‌توان یک **تابع مستقل**، یک **کلاس** دیگر (همه توابع کلاس) و یا یک **تابع عضو از یک کلاس** دیگر را دوست یک کلاس اعلان کرد.
- برای معرفی یک تابع دوست کافی است که prototype تابع را همراه با کلمه کلیدی «**friend**» را در تعریف کلاس قرار داد.

friend return_type Function_name (parameters_list);

- برای معرفی همه توابع یک کلاس به عنوان دوست کلاس دیگر کافی است نام کلاس را همراه کلمه کلیدی «**friend**» و کلمه کلیدی «**class**» را در تعریف کلاس قرار داد.

friend class class_name;

- چون تابع دوست جزء توابع کلاس به حساب نمی‌آید، بنابراین سطح دسترسی برای آن بی معنی است و می‌توان اعلان دوستی را در هر جای بدنه کلاس قرار داد؛ ولی معمولاً اعلان دوستیها در ابتدای کلاس و قبل از آوردن هر نوع سطح دسترسی قرار می‌گیرد.



مثال ۱: تابع دوست (Point.h)

```
1  #ifndef POINT_H
2  #define POINT_H
3
4
5  class Point
6  {
7      friend Point ADD(const Point &, const Point &);
8  public:
9      Point(int = 0, int = 0);
10     ~Point();
11
12     int Getx() const { return x; }
13     void Setx(int val) { x = val; }
14     int Gety() const { return y; }
15     void Sety(int val) { y = val; }
16     void display() const;
17
18     protected:
19
20     private:
21         int x;
22         int y;
23 };
24
25 #endif // POINT_H
26
```

اعلان تابع دوست ←



مثال ۱: تابع دوست (Point.cpp)

```
1  #include <iostream>
2  #include "Point.h"
3
4  using namespace std;
5
6  Point::Point(int a, int b): x(a), y(b)
7  {
8      cout << "(" << x << ", " << y << ")" is constructed!" << endl;
9  }
10
11  Point::~~Point()
12  {
13      cout << "(" << x << ", " << y << ")" is destructed!" << endl;
14  }
15
16  void Point::display() const
17  {
18      cout << "(" << x << ", " << y << ")" << endl;
19  }
20
```



مثال ۱: تابع دوست (فایل main.cpp)

```
1  #include <iostream>
2  #include "Point.h"
3
4  using namespace std;
5
6  Point ADD(const Point &, const Point &);
7
8  int main()
9  {
10     cout << "begin of main" << endl;
11     Point point1(100,200), point2(10,20), sum;
12     sum = ADD(point1, point2);
13     sum.display();
14
15     cout << "end of main" << endl;
16
17     return 0;
18 }
19
20 Point ADD(const Point &p1, const Point &p2)
21 {
22     cout << "begin of ADD" << endl;
23     Point p3;
24
25     p3.x = p1.x + p2.x;
26     p3.y = p1.y + p2.y;
27     //p1.x = 1000; p2.Setx(1000); تغییر دادن عضو داده ای ثابت مجاز نیست
28
29     p1.display();
30     p2.display();
31     p3.display();
32
33     cout << "end of ADD" << endl;
34     return p3;
35 }
36
```

```
"C:\Users\Maryam\courses\Advanced programming\Point2\b...
begin of main
(100,200) is constructed!
(10,20) is constructed!
(0,0) is constructed!
begin of ADD
(0,0) is constructed!
(100,200)
(10,20)
(110,220)
end of ADD
(110,220) is destructed!
(110,220)
end of main
(110,220) is destructed!
(10,20) is destructed!
(100,200) is destructed!

Process returned 0 (0x0) execution time : 0.270 s
Press any key to continue.
```

دسترسی به اعضای خصوصی کلاس ←

مثال ۲: پیاده‌سازی تابع ADD به عنوان تابع عضو

```
17 string sInput;  
18 int iLength, iN;  
19 double dblTemp;  
20 bool again = true;  
  
21 while (again) {  
22     iN = -1;  
23     again = false;  
24     getline(cin, sInput);  
25     system("cls");  
26     stringstream(sInput) >> dblTemp;  
27     iLength = sInput.length();  
28     if (iLength < 4) {  
29         again = true;  
30         continue;  
31     } else if (sInput[iLength - 3] != '.') {  
32         again = true;  
33     }  
34 }
```



مثال ۳: بررسی زمان فراخوانی سازنده و نابودکننده (فایل Point.h)

```
1  #ifndef POINT_H
2  #define POINT_H
3
4
5  class Point
6  {
7      friend void func(Point&); ← اعلان تابع دوست
8  public:
9      Point(int = 0, int = 0);
10     ~Point();
11
12     int Getx() const { return x; }
13     void Setx(int val) { x = val; }
14     int Gety() const { return y; }
15     void Sety(int val) { y = val; }
16     void display() const;
17
18     protected:
19
20     private:
21         int x;
22         int y;
23     };
24
25
26 #endif // POINT_H
27
```



مثال ۳: بررسی زمان فراخوانی سازنده و نابودکننده (فایل Point.cpp)

```
1  #include <iostream>
2  #include "Point.h"
3
4  using namespace std;
5
6  Point::Point(int a, int b):x(a),y(b)
7  {
8      cout << "("<<x<<','<<y<<") is constructed!"<<endl;
9  }
10
11  Point::~~Point()
12  {
13      cout << "("<<x<<','<<y<<") is destructed!"<<endl;
14  }
15
16  void Point::display() const
17  {
18      cout << '(' << x << ',' << y << ')' << endl;
19  }
20
```



مثال ۳: بررسی زمان فراخوانی سازنده و نابودکننده (فایل main.cpp)

```
42 void func(Point &p1)
43 {
44     cout << "\nFUNC FUNCTION: EXECUTION BEGINS" << endl;
45     Point p2(101,102);
46     static Point p3(111,112);
47     p1.x++; p1.y++;
48     p2.x++; p2.y++;
49     p3.x++; p3.y++;
50     GlobalPoint.x++; GlobalPoint.y++;
51
52     cout << " Local Variable in func:"; p2.display();
53     cout << " Local Static Variable in func:"; p3.display();
54
55     cout << "\nFUNC FUNCTION: EXECUTION ENDS" << endl;
56 }
57
```



مثال ۳: بررسی زمان فراخوانی سازنده و نابودکننده (فایل main.cpp)

```
1  #include <iostream>
2  #include "Point.h"
3
4  using namespace std;
5
6  Point GlobalPoint(1,2);
7
8  void func(Point&);
9
10 int main()
11 {
12     cout << "\nMAIN FUNCTION: EXECUTION BEGINS" << endl;
13
14     Point point1(11,12);
15     static Point point2(21,22);
16     cout << "Global Variable before main:";GlobalPoint.display();
17     cout << "Local Variable in main:";point1.display();
18     cout << "Local Static Variable in main:";point2.display();
19
20     func(point1);
21
22     cout << "\nMAIN FUNCTION: EXECUTION RESUMES" << endl;
23     cout << "Global Variable after first function call:";GlobalPoint.display();
24     cout << "Local Variable in main after first function call:";point1.display();
25     cout << "Local Static Variable in main after first function call:";point2.display();
26
27     func(point1);
28
29     cout << "\nMAIN FUNCTION: EXECUTION RESUMES" << endl;
30     cout << "Global Variable after second function call:";GlobalPoint.display();
31     cout << "Local Variable in main after second function call:";point1.display();
32     cout << "Local Static Variable in main after second function call:";point2.display();
33
34 }
35
36 Point point3(31,32);
37 cout << "Local Variable in main:";point3.display();
38
39 cout << "\nMAIN FUNCTION: EXECUTION ENDS" << endl;
40 return 0;
```

```
"C:\Users\Maryam\courses\Advanced programming\Point2\b...
(1,2) is constructed!
MAIN FUNCTION: EXECUTION BEGINS
(11,12) is constructed!
(21,22) is constructed!
Global Variable before main:(1,2)
Local Variable in main:(11,12)
Local Static Variable in main:(21,22)
FUNC FUNCTION: EXECUTION BEGINS
(101,102) is constructed!
(111,112) is constructed!
Local Variable in func:(102,103)
Local Static Variable in func:(112,113)
FUNC FUNCTION: EXECUTION ENDS
(102,103) is destructed!
MAIN FUNCTION: EXECUTION RESUMES
Global Variable after first function call:(2,3)
Local Variable in main after first function call:(12,13)
Local Static Variable in main after first function call:(21,22)
FUNC FUNCTION: EXECUTION BEGINS
(101,102) is constructed!
Local Variable in func:(102,103)
Local Static Variable in func:(113,114)
FUNC FUNCTION: EXECUTION ENDS
(102,103) is destructed!
MAIN FUNCTION: EXECUTION RESUMES
Global Variable after second function call:(3,4)
Local Variable in main after second function call:(13,14)
Local Static Variable in main after second function call:(21,22)
(13,14) is destructed!
(31,32) is constructed!
Local Variable in main:(31,32)
MAIN FUNCTION: EXECUTION ENDS
(31,32) is destructed!
(113,114) is destructed!
(21,22) is destructed!
(3,4) is destructed!
Process returned 0 (0x0)   execution time : 0.611 s
Press any key to continue.
```

اعضای ایستا در کلاس

- قبلا گفتیم که هر شی کپی خودش را از اعضای داده‌ای کلاس دارد. اما ممکن است ما نیاز به داده‌ای داشته باشیم که بین همه اشیای یک کلاس مشترک باشد و مختص یک نمونه از کلاس نباشد.
- به عنوان نمونه فرض کنید که به هر علتی ما بخواهیم تعداد نمونه‌هایی که از یک کلاس ایجاد کردیم بدانیم، این اطلاعات مربوط به یک شی خاص نیست و مرتبط به همه نمونه‌های کلاس است؛ به عبارت دیگر این داده در حوزه کلاس معنا پیدا می‌کند و نه شی!
- برای داده‌هایی ازین دست در کلاسها، از اعضای داده‌ای **ایستا** استفاده می‌شود. آوردن کلمه کلیدی **static** قبل از نوع داده و نام متغیر می‌تواند آن داده را ایستا معرفی کند. نکته مهم در مورد اعضای داده‌ای ایستا در کلاس این است که این داده‌ها قبل از ایجاد هر نمونه‌ای از کلاس وجود دارند.
- عضو ایستای کلاس می‌تواند مانند بقیه اعضا دارای سطح دسترسی **عمومی**، **خصوصی** و **حفاظت شده** باشد.
- مقداردهی اولیه یک عضو داده‌ای ایستا متفاوت از بقیه اعضای داده‌ای کلاس است و **با استفاده از تابع سازنده نیست**.
- برای اعضای داده‌ای ایستای کلاس، بدون ایجاد شی حافظه گرفته می‌شود و با اعلان آن خارج از بدنه کلاس **مشابه متغیرهای سراسری** ایجاد و مقداردهی اولیه می‌شوند.
- اعضای داده‌ای (صحیح) **ثابت ایستا** می‌توانند در تعریف کلاس ایجاد و مقداردهی شوند.
- اعضای داده‌ای ایستای پایه (نوع داده‌های اولیه) به صورت پیش‌فرض **صفر** در نظر گرفته می‌شوند.



توابع ایستای کلاس

- به اعضای ایستای عمومی می توان بدون ایجاد نمونه ای از کلاس و با استفاده از نام کلاس و عملگر :: دسترسی پیدا کرد و در صورت نیاز مقدار آن را تغییر داد.

class_name::static_variable_name

- اعضای ایستای خصوصی فقط در توابع عضو کلاس و همچنین توابع دوست قابل دسترسی هستند. برای دسترسی به اعضای ایستای خصوصی بدون ایجاد نمونه ای از کلاس از توابع ایستا استفاده می شود.
- برای ایجاد توابع ایستا در کلاس، کلمه کلیدی **static** قبل از prototype تابع آورده می شود. تنها توابعی می توانند **ایستا** معرفی شوند که در آنها نه به اعضای غیرایستا مراجعه شود و نه توابع غیرایستای کلاس فراخوانی شود.
- توابع ایستا را نیز می توان بدون ایجاد نمونه ای از کلاس و با استفاده از نام کلاس، عملگر :: و نام تابع ایستا فراخوانی کرد.

class_name::static_memberFunction_name(...)

- هنگام ایجاد و مقداردهی اولیه اعضای داده ای ایستا خارج از بدنه کلاس و همچنین هنگام پیاده سازی توابع ایستا، خارج از بدنه کلاس کلمه کلیدی **static** آورده نمی شود.
- **داده های ایستای** کلاس را می توان به صورت ثابت تعریف کرد؛ ولی **توابع ایستا** را نمی توان ثابت تعریف کرد. زیرا توابع ایستا مستقل از اشیاء وجود دارند و نیازی به فراخوانی آنها با استفاده از شی ثابت نیست.



مثال ۴: عضو داده‌ای ایستای عمومی در کلاس (Circle.h)

```
1  #ifndef CIRCLE_H
2  #define CIRCLE_H
3
4
5  class Circle
6  {
7      public:
8
9          Circle(int = 1); // تابع سازنده
10         ~Circle(); // تابع سازنده
11         void setRadius(int r); // تابع مقداردهی به شعاع
12         int getRadius(); // تابع دستیابی به مقدار شعاع
13         float getPerimeter(); // تابع محاسبه محیط
14         float getArea(); // تابع محاسبه مساحت
15         static double PI; ← اعلان عضو عمومی ایستا
16
17     private:
18         int Radius;
19 };
20
21 #endif // CIRCLE_H
22
23
```



مثال ۴: عضو داده‌ای ایستای عمومی در کلاس (فایل Circle.cpp)

```
1  #include "Circle.h"
2  #include <iostream>
3  using namespace std;
4
5  double Circle::PI = 3.14; ← ایجاد و مقداردهی اولیه‌ی عضو ایستای کلاس
6
7  Circle::Circle(int r)
8  {
9      setRadius(r);
10 }
11
12
13 Circle::~~Circle()
14 {
15     cout << "circle was destructed! Radius =" << Radius << endl;
16 }
17
18 void Circle::setRadius(int r)
19 {
20     if (r > 0)
21         Radius = r;
22     else
23     {
24         cout << "Radius is invalid! Set Radius = 1" << endl;
25         Radius = 1;
26     }
27 }
28
29 int Circle::getRadius()
30 {
31     return Radius;
32 }
33
34 float Circle::getPrimeter()
35 {
36     return 2 * Radius * PI;
37 }
38
39 float Circle::getArea()
40 {
41     return Radius * Radius * PI;
```

استفاده از عضو ایستا در توابع کلاس



مثال ۴: عضو داده‌ای ایستای عمومی در کلاس (فایل main.cpp)

```
1  #include <iostream>
2  #include "Circle.h"
3
4
5  using namespace std;
6
7
8  int main()
9  {
10     cout << "begin of main" << endl;
11     cout << "PI of Circle class is " << Circle::PI << endl;
12
13     Circle c1(10), c2, circles[5]={100,200,300};
14
15
16     cout << "-----\n";
17     cout << "PI of c1 is " << c1.PI << endl;
18     cout << "-----\n";
19     cout << "PI of c2 is " << c2.PI << endl;
20     cout << "-----\n";
21     for (int i = 0; i<5 ; i++)
22     {
23         cout<< "PI of circels[" << i << "]" is " << circles[i].PI << endl;
24     }
25     cout << "-----\n";
26
27     Circle::PI = 3.1415;|
28
29     cout << "PI of Circle class is " << Circle::PI << endl;
30     cout << "-----\n";
31     cout << "PI of c1 is " << c1.PI << endl;
32     cout << "-----\n";
33     cout << "PI of c2 is " << c2.PI << endl;
34     cout << "-----\n";
35     for (int i = 0; i<5 ; i++)
36     {
37         cout<< "PI of circels[" << i << "]" is " << circles[i].PI << endl;
38     }
39     cout << "-----\n";
40
41     cout << "end of main" <<endl;
42
43     return 0;
44 }
45
```

```
"C:\Users\Maryam\courses\Advanced programing\program...
begin of main
PI of Circle class is 3.14
-----
PI of c1 is 3.14
-----
PI of c2 is 3.14
-----
PI of circels[0] is 3.14
PI of circels[1] is 3.14
PI of circels[2] is 3.14
PI of circels[3] is 3.14
PI of circels[4] is 3.14
-----
PI of Circle class is 3.1415
-----
PI of c1 is 3.1415
-----
PI of c2 is 3.1415
-----
PI of circels[0] is 3.1415
PI of circels[1] is 3.1415
PI of circels[2] is 3.1415
PI of circels[3] is 3.1415
PI of circels[4] is 3.1415
-----
end of main
circle was destructed! Radius =1
circle was destructed! Radius =1
circle was destructed! Radius =300
circle was destructed! Radius =200
circle was destructed! Radius =100
circle was destructed! Radius =1
circle was destructed! Radius =10

Process returned 0 (0x0)   execution time : 0.330 s
Press any key to continue.
```



مثال ۵: عضو داده‌ای ایستای خصوصی و توابع ایستا در کلاس (فایل Point.h)

```
1  #ifndef POINT_H
2  #define POINT_H
3
4
5  class Point
6  {
7      friend void func(Point);
8      public:
9          Point(int = 0, int = 0);
10         Point(const Point&);
11         ~Point();
12
13         int Getx() const { return x; }
14         void Setx(int val) { x = val; }
15         int Gety() const { return y; }
16         void Sety(int val) { y = val; }
17         void display() const;
18         static unsigned int GetCount(); ← تابع ایستا
19
20     protected:
21
22     private:
23         int x;
24         int y;
25         static unsigned int count; ← عضو داده‌ای خصوصی ایستا
26     };
27
28 #endif // POINT_H
29
```



مثال ۵: عضو داده‌ای ایستای خصوصی و توابع ایستا در کلاس (فایل Point.cpp)

```
1  #include <iostream>
2  #include "Point.h"
3
4  using namespace std;
5
6  unsigned int Point::count = 0;    ← مقداردهی به عضو داده‌ای ایستا
7
8  unsigned int Point::GetCount()    ← پیاده‌سازی تابع ایستا
9  {
10     return count;
11 }
12
13 Point::Point(int a, int b):x(a),y(b)
14 {
15     cout << " Default/Parameterized constructor: ("<<x<<','<<y<<") is constructed!"<<endl;
16     count++;
17 }
18
19 Point::Point(const Point &p)
20 {
21     x = p.x;
22     y = p.y;
23     cout << " Copy constructor: ("<<x<<','<<y<<") is constructed!"<<endl;
24     count++;
25 }
26
27 Point::~Point()
28 {
29     cout << " Destructor: ("<<x<<','<<y<<") is destructed!"<<endl;
30     count--;
31 }
32
33 void Point::display() const
34 {
35     cout << '(' << x << ',' << y << ')' << endl;
36 }
```



مثال ۵: عضو داده‌ای ایستای خصوصی و توابع ایستا در کلاس (فایل main.cpp)

- نکته ۱: هر کدام از خطوطی که در کد به صورت توضیحات در آمده‌اند (خطوط ۵۴-۶۲)، می‌توانند جایگزین دستور خط ۵۳ شوند.
- نکته ۲: علت دسترسی مستقیم به متغیر خصوصی کلاس Point (count) این است که تابع func در کلاس Point به عنوان دوست معرفی شده است (به اسلاید ۱۶ (Point.h) مراجعه شود).

```
47 void func(Point p1)
48 {
49     cout << "FUNC FUNCTION: EXECUTION BEGINS" << endl;
50     Point p2(101,102);
51     static Point p3(111,112);
52
53     cout << "Number of points is "<< Point::count << " (using class name and static private class member)" << endl;
54     // cout << "Number of points is "<< GlobalPoint.count << " (using global object name and private class member)" << endl;
55     // cout << "Number of points is "<< p1.count << " (using reference object name in func and private class member)" << endl;
56     // cout << "Number of points is "<< p2.count << " (using local object name in func and private class member)" << endl;
57     // cout << "Number of points is "<< p3.count << " (using static local object name in func and private class member)" << endl;
58     // cout << "Number of points is "<< Point::GetCount() << " (using class name and static function call)" << endl;
59     // cout << "Number of points is "<< GlobalPoint.GetCount() << " (using global object name and static function call)" << endl;
60     // cout << "Number of points is "<< p1.GetCount() << " (using reference object name in func and static function call)" << endl;
61     // cout << "Number of points is "<< p2.GetCount() << " (using local object name in func and static function call)" << endl;
62     // cout << "Number of points is "<< p3.GetCount() << " (using static local object name in func and static function call)" << endl;
63
64     cout << "FUNC FUNCTION: EXECUTION ENDS" << endl;
65 }
```

دسترسی به متغیر ایستا
با استفاده از
نام کلاس و نام اشیا

فراخوانی تابع ایستا
با استفاده از
نام کلاس و نام اشیا



مثال ۵: عضو داده‌ای ایستای خصوصی و توابع ایستا در کلاس (فایل main.cpp)

- **نکته ۱:** هر کدام از خطوطی که در کد به صورت توضیحات در آمده‌اند، می‌توانند جایگزین دستور فراخوانی تابع ایستا شوند. فراخوانی تابع ایستا می‌تواند هم از طریق نام کلاس و هم از طریق نام اشیای قابل دسترسی انجام شود.
- **نکته ۲:** در تابع main امکان دسترسی مستقیم به متغیر خصوصی کلاس Point (count) وجود ندارد.

```
1  #include <iostream>
2  #include "Point.h"
3
4  using namespace std;
5
6  Point GlobalPoint(1,2);
7
8  void func(Point);
9
10 int main()
11 {
12     cout << "\nMAIN FUNCTION: EXECUTION BEGINS" << endl;
13     cout << "Number of points before instantiation of any objects in main is "
14     << Point::GetCount() << " (using class name and static function call)" << endl;
15     // cout << "Number of points is " << GlobalPoint.GetCount() << " (using global object name and static function call)" << endl;
16
17     Point point1(11,12);
18     static Point point2(21,22);
19
20     cout << "Number of points is " << Point::GetCount() << " (using class name and static function call)" << endl;
21     // cout << "Number of points is " << GlobalPoint.GetCount() << " (using global object name and static function call)" << endl;
22     // cout << "Number of points is " << point1.GetCount() << " (using local object name in main and static function call)" << endl;
23     // cout << "Number of points is " << point2.GetCount() << " (using static local object name in main and static function call)" << endl;
24     cout << endl;
25     func(point1);
26     cout << "\nMAIN FUNCTION: EXECUTION RESUMES" << endl;
27     // cout << "Number of points is " << Point::GetCount() << " (using class name and static function call)" << endl;
28     // cout << "Number of points is " << GlobalPoint.GetCount() << " (using global object name and static function call)" << endl;
29     cout << "Number of points is " << point1.GetCount() << " (using local object name in main and static function call)" << endl;
30     // cout << "Number of points is " << point2.GetCount() << " (using static local object name in main and static function call)" << endl;
31
32     cout << endl;
33     func(point1);
34     cout << "\nMAIN FUNCTION: EXECUTION RESUMES" << endl;
35     // cout << "Number of points is " << Point::GetCount() << " (using class name and static function call)" << endl;
36     // cout << "Number of points is " << GlobalPoint.GetCount() << " (using global object name and static function call)" << endl;
37     // cout << "Number of points is " << point1.GetCount() << " (using local object name in main and static function call)" << endl;
38     cout << "Number of points is " << point2.GetCount() << " (using static local object name in main and static function call)" << endl;
39 }
40 // cout << "Number of points is " << Point::GetCount() << " (using class name and static function call)" << endl;
41 cout << "Number of points is " << GlobalPoint.GetCount() << " (using global object name and static function call)" << endl;
42
43 cout << "\nMAIN FUNCTION: EXECUTION ENDS" << endl;
44 return 0;
45 }
```



مثال ۵: عضو داده‌ای ایستای خصوصی و توابع ایستا در کلاس (فایل main.cpp)

```
1  #include <iostream>
2  #include "Point.h"
3
4  using namespace std;
5
6  Point GlobalPoint(1,2);
7
8  void func(Point);
9
10 int main()
11 {
12     cout << "\nMAIN FUNCTION: EXECUTION BEGINS" << endl;
13     cout << "Number of points before instantiation of any objects in main is "
14     << Point::GetCount() << " (using class name and static function call)" << endl;
15     // cout << "Number of points is " << GlobalPoint.GetCount() << " (using global object name and s
16 {
17     Point point1(11,12);
18     static Point point2(21,22);
19
20     cout << "Number of points is " << Point::GetCount() << " (using class name and static function
21     // cout << "Number of points is " << GlobalPoint.GetCount() << " (using global object name and
22     // cout << "Number of points is " << point1.GetCount() << " (using local object name in main a
23     // cout << "Number of points is " << point2.GetCount() << " (using static local object name in
24     cout << endl;
25     func(point1);
26     cout << "\nMAIN FUNCTION: EXECUTION RESUMES" << endl;
27     // cout << "Number of points is " << Point::GetCount() << " (using class name and static func
28     // cout << "Number of points is " << GlobalPoint.GetCount() << " (using global object name and
29     cout << "Number of points is " << point1.GetCount() << " (using local object name in main and
30     // cout << "Number of points is " << point2.GetCount() << " (using static local object name in
31
32     cout << endl;
33     func(point1);
34     cout << "\nMAIN FUNCTION: EXECUTION RESUMES" << endl;
35     // cout << "Number of points is " << Point::GetCount() << " (using class name and static func
36     // cout << "Number of points is " << GlobalPoint.GetCount() << " (using global object name and
37     // cout << "Number of points is " << point1.GetCount() << " (using local object name in main a
38     cout << "Number of points is " << point2.GetCount() << " (using static local object name in m
39 }
40 // cout << "Number of points is " << Point::GetCount() << " (using class name and static functi
41 cout << "Number of points is " << GlobalPoint.GetCount() << " (using global object name and s
42
43 cout << "\nMAIN FUNCTION: EXECUTION ENDS" << endl;
44 return 0;
45 }
```

```
"C:\Users\Maryam\courses\Advanced programming\Point2\b...
Default/Parameterized constructor: (1,2) is constructed!

MAIN FUNCTION: EXECUTION BEGINS
Number of points before instantiation of any objects in main is 1 (using class n
ame and static function call)
Default/Parameterized constructor: (11,12) is constructed!
Default/Parameterized constructor: (21,22) is constructed!
Number of points is 3 (using class name and static function call)

Copy constructor: (11,12) is constructed!
FUNC FUNCTION: EXECUTION BEGINS
Default/Parameterized constructor: (101,102) is constructed!
Default/Parameterized constructor: (111,112) is constructed!
Number of points is 6 (using class name and static private class member)
FUNC FUNCTION: EXECUTION ENDS
Destructor: (101,102) is destructed!
Destructor: (11,12) is destructed!

MAIN FUNCTION: EXECUTION RESUMES
Number of points is 4 (using local object name in main and static function call)

Copy constructor: (11,12) is constructed!
FUNC FUNCTION: EXECUTION BEGINS
Default/Parameterized constructor: (101,102) is constructed!
Number of points is 6 (using class name and static private class member)
FUNC FUNCTION: EXECUTION ENDS
Destructor: (101,102) is destructed!
Destructor: (11,12) is destructed!

MAIN FUNCTION: EXECUTION RESUMES
Number of points is 4 (using static local object name in main and static functio
n call)
Destructor: (11,12) is destructed!
Number of points is 3 (using global object name and static function call)

MAIN FUNCTION: EXECUTION ENDS
Destructor: (111,112) is destructed!
Destructor: (21,22) is destructed!
Destructor: (1,2) is destructed!

Process returned 0 (0x0)   execution time : 0.094 s
Press any key to continue.
```

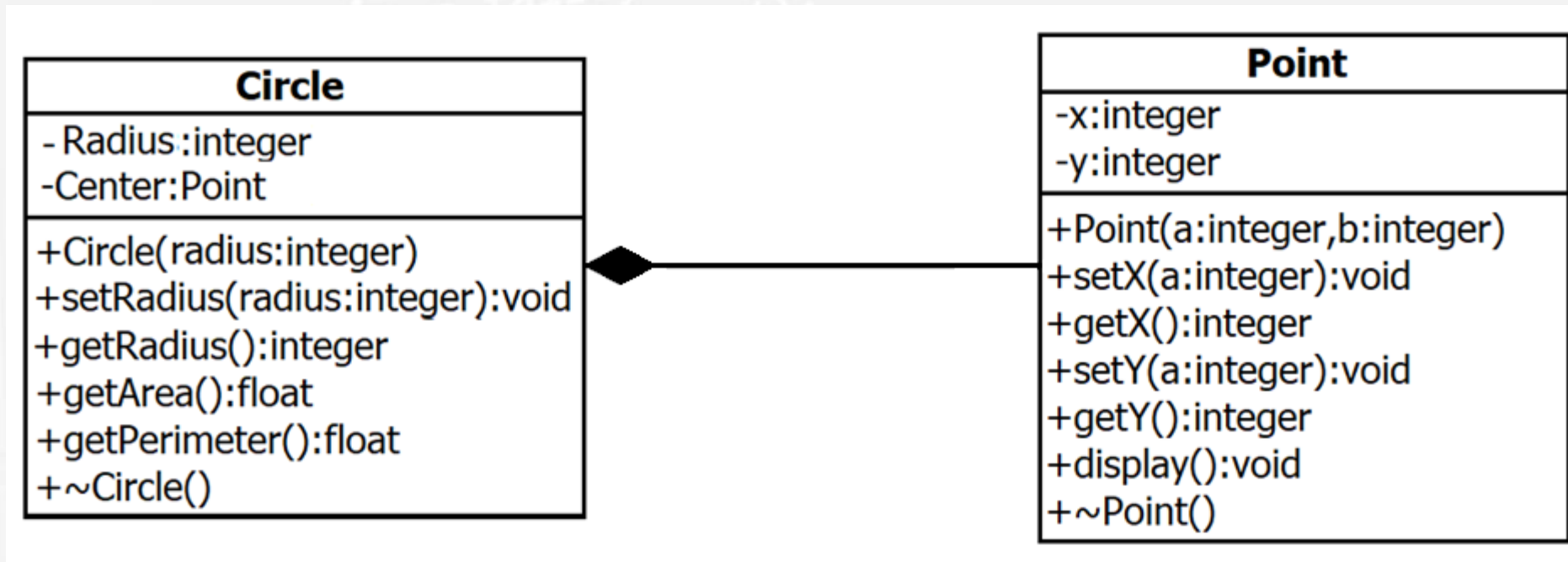
ترکیب: اشیای عضو کلاس

- گاهی اوقات نیاز داریم در یک کلاس، شئی از کلاس دیگر را به عنوان عضو داده‌ای داشته باشیم. به این مفهوم **ترکیب (composition)** (یا در برخی موارد تجمیع (aggregation)) کلاسها می‌گویند.
- این ارتباط بین دو کلاس را **رابطه has-a** نیز می‌نامند؛ یعنی یک کلاس **یک شی** (از کلاس دیگر) **دارد**. (این نامگذاری در مقابل ارتباط وراثت است که به رابطه **is-a** مشهور است).
- داشتن شئی به عنوان عضو کلاس مجوز دسترسی به اعضای خصوصی شی را نمی‌دهد.
- هنگام ایجاد شئی از کلاس و فراخوانی سازنده، سازنده اشیای موجود در کلاس نیز فراخوانی می‌شود. می‌توان صریحا سازنده را با ارسال پارامترهای لازم فراخوانی کرد. در صورت عدم انجام این کار، سازنده پیش‌فرض (بدون پارامتر) فراخوانی می‌شود و در صورت عدم وجود سازنده پیش‌فرض کامپایلر خطا ایجاد خواهد کرد.
- در صورت ارسال شئی از نوع شی عضو به سازنده شی عضو، مانند انتساب اشیای به یکدیگر اعضای داده‌ای نظیر به نظیر کپی خواهند شد.
- هنگام از بین رفتن شی نیز نابودکننده‌های اشیای درون کلاس فراخوانی می‌شود.
- **ترتیب فراخوانی سازنده‌ها و نابودکننده‌ها کلاس اصلی و اشیای درون آن به چه شکلی خواهد بود؟**



ترکیب: اشیای عضو کلاس

- مثال ۶: می‌خواهیم به کلاس **Circle** که قبلاً ایجاد کردیم عضو داده‌ای مختصات مرکز دایره را اضافه کنیم. این عضو از نوع کلاس **Point** خواهد بود.



مثال ۶: ترکیب (فایل Point.h)

```
1  #ifndef POINT_H
2  #define POINT_H
3
4
5  class Point
6  {
7      friend void func(Point);
8      public:
9          Point(int = 0, int = 0);
10         Point(const Point&);
11         ~Point();
12
13         int Getx() const { return x; }
14         void Setx(int val) { x = val; }
15         int Gety() const { return y; }
16         void Sety(int val) { y = val; }
17         void display() const;
18         static unsigned int GetCount();
19
20     protected:
21
22     private:
23         int x;
24         int y;
25         static unsigned int count;
26 };
27
28 #endif // POINT_H
29
```



مثال ٦: ترکیب (فایل Point.cpp)

```
1  #include <iostream>
2  #include "Point.h"
3
4  using namespace std;
5
6  unsigned int Point::count = 0;
7
8  unsigned int Point::GetCount()
9  {
10     return count;
11 }
12
13 Point::Point(int a, int b):x(a),y(b)
14 {
15     cout << " ("<<x<<','<<y<<") is constructed!"<<endl;
16     count++;
17 }
18
19 Point::Point(const Point &p)
20 {
21     x = p.x;
22     y = p.y;
23     cout << " ("<<x<<','<<y<<") is constructed!"<<endl;
24     count++;
25 }
26
27 Point::~~Point()
28 {
29     cout << " ("<<x<<','<<y<<") is destructed!"<<endl;
30     count--;
31 }
32
33 void Point::display() const
34 {
35     cout << '(' << x <<','<< y<<')'<<endl;
36 }
```



مثال ۶: ترکیب (فایل Circle.h)

```
1  #ifndef CIRCLE_H
2  #define CIRCLE_H
3
4  #include "Point.h" ← ضمیمه کردن تعریف کلاس Point
5
6  class Circle
7  {
8      public:
9
10         Circle(int = 1, int = 1, int = 1); } توابع سازنده تغییر یافته کلاس Circle
11         Circle(Point &, int = 1);
12         ~Circle();
13         void SetRadius(int r);
14         int GetRadius() const;
15         void SetCenter(Point &); } توابع مقداره‌ی و دستیابی به شی Center
16         Point GetCenter() const;
17         float GetPerimeter() const;
18         float GetArea() const;
19         static double PI;
20
21     private:
22         int Radius;
23         Point Center; ← اعلان شی از کلاس Point در کلاس Circle
24     };
25
26 #endif // CIRCLE_H
27
28
```



مثال ۶: ترکیب (فایل Circle.cpp)

```
1  #include "Circle.h"
2  #include <iostream>
3  using namespace std;
4
5  double Circle::PI = 3.14;
6
7  Circle::Circle(int x,int y,int r):Center(x,y)
8  {
9      SetRadius(r);
10     cout << "circle was constructed! Radius = " << Radius << ",Center = ";
11     Center.display();
12 }
13
14 Circle::Circle(Point &p,int r):Center(p)
15 {
16     SetRadius(r);
17     cout << "circle was constructed! Radius = " << Radius << ",Center = ";
18     Center.display();
19 }
20
21 Circle::~Circle()
22 {
23     cout << "circle was desstructured! Radius = " << Radius << ",Center = ";
24     Center.display();
25 }
26
27 void Circle::SetCenter(Point &p)
28 {
29     Center.SetX(p.GetX());
30     Center.SetY(p.GetY());
31     // Center = p;
32 }
33
34 Point Circle::GetCenter() const
35 {
36     return Center;
37 }
38
39
```

← سازنده کلاس Circle
(فراخوانی سازنده کلاس Point و ارسال پارامتر)

← سازنده کلاس Circle
(فراخوانی سازنده کپی کلاس Point و ارسال شیء)

← تابع مقداردهی شیء Center

← تابع دستیابی شیء Center



مثال ۶: ترکیب (ادامه فایل Circle.cpp)

```
40 void Circle::SetRadius(int r)
41 {
42     if (r > 0)
43         Radius = r;
44     else
45     {
46         cout << "Radius is invalid! Set Radius = 1" << endl;
47         Radius = 1;
48     }
49 }
50
51 int Circle::GetRadius() const
52 {
53     return Radius;
54 }
55
56 float Circle::GetPrimeter() const
57 {
58     return 2 * Radius * PI;
59 }
60
61 float Circle::GetArea() const
62 {
63     return Radius * Radius * PI;
64 }
65
```



```

1  #include <iostream>
2  #include "Point.h"
3  #include "Circle.h"
4
5  using namespace std;
6
7  int main()
8  {
9      cout << "\nMAIN FUNCTION: EXECUTION BEGINS" << endl;
10     Point point1(10,20),point2;
11     Circle circle1(100,200,50),circle2(70),circle3(point1,12);
12     Circle circle4(point2),Circles[5];
13     point2 = circle2.GetCenter();
14     cout << " Center of circle2 is ";
15     point2.display();
16     cout << " Center of circle 1: x = " << circle1.GetCenter().GetX()
17         << ",y = " << circle1.GetCenter().GetY() << endl;
18     for(int i=0;i<5;i++)
19     {
20         int x,y,r;
21         cout << " Enter center and radius of circles[" << i<<"]:";
22         cin >> x >> y >> r;
23         point2.SetX(x);
24         point2.SetY(y);
25         Circles[i].SetCenter(point2);
26         Circles[i].SetRadius(r);
27     }
28     cout << "\nMAIN FUNCTION: EXECUTION ENDS" << endl;
29     return 0;
30 }
31

```

```

"C:\Users\Maryam\courses\Advanced programing\Composi...
MAIN FUNCTION: EXECUTION BEGINS
(10,20) is constructed!
(0,0) is constructed!
(100,200) is constructed!
circle was constructed! Radius = 50,Center = (100,200)
(70,1) is constructed!
circle was constructed! Radius = 1,Center = (70,1)
circle was constructed! Radius = 12,Center = (10,20)
circle was constructed! Radius = 1,Center = (0,0)
(1,1) is constructed!
circle was constructed! Radius = 1,Center = (1,1)
(1,1) is constructed!
circle was constructed! Radius = 1,Center = (1,1)
(1,1) is constructed!
circle was constructed! Radius = 1,Center = (1,1)
(1,1) is constructed!
circle was constructed! Radius = 1,Center = (1,1)
(70,1) is destroyed!
Center of circle2 is (70,1)
Center of circle 1: x = 100,y = 200
(100,200) is destroyed!
(100,200) is destroyed!
Enter center and radius of circles[0]:10 11 12
Enter center and radius of circles[1]:20 21 22
Enter center and radius of circles[2]:30 31 32
Enter center and radius of circles[3]:40 41 42
Enter center and radius of circles[4]:50 51 52

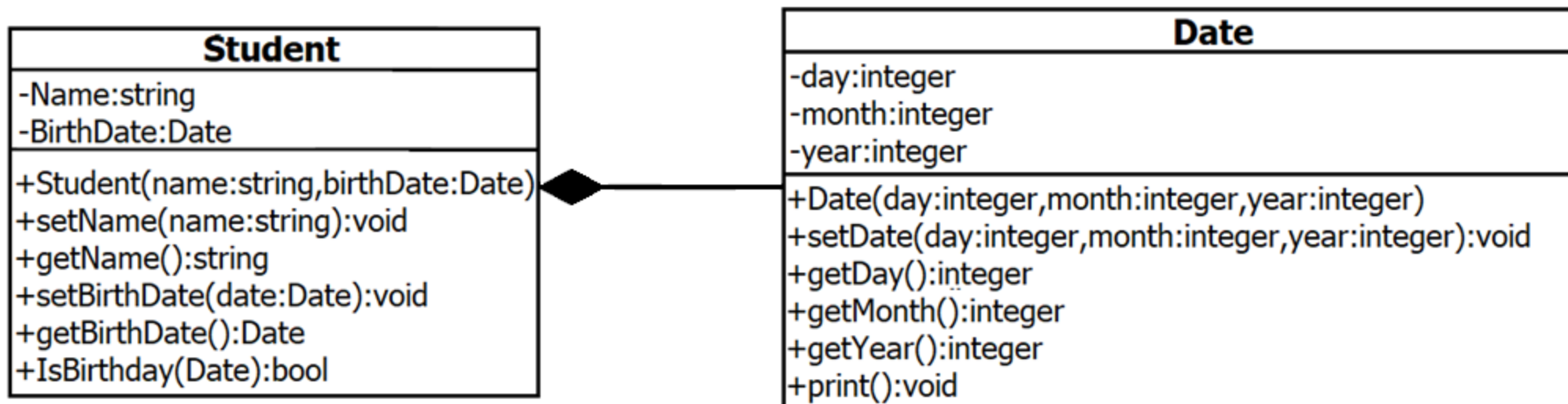
MAIN FUNCTION: EXECUTION ENDS
circle was desstruced! Radius = 52,Center = (50,51)
(50,51) is destroyed!
circle was desstruced! Radius = 42,Center = (40,41)
(40,41) is destroyed!
circle was desstruced! Radius = 32,Center = (30,31)
(30,31) is destroyed!
circle was desstruced! Radius = 22,Center = (20,21)
(20,21) is destroyed!
circle was desstruced! Radius = 12,Center = (10,11)
(10,11) is destroyed!
circle was desstruced! Radius = 1,Center = (0,0)
(0,0) is destroyed!
circle was desstruced! Radius = 12,Center = (10,20)
(10,20) is destroyed!
circle was desstruced! Radius = 1,Center = (70,1)
(70,1) is destroyed!
circle was desstruced! Radius = 50,Center = (100,200)
(100,200) is destroyed!
(50,51) is destroyed!
(10,20) is destroyed!

Process returned 0 (0x0)   execution time : 42.024 s
Press any key to continue.

```

مرور مطالب

- مثال ۷: می‌خواهیم نام و تاریخ تولد دانشجویان یک کلاس را گردآوری کرده و با دریافت تاریخ روز به کسانی که تولد آنها امروز است تبریک بگوییم.



مثال ۷: (فایل Date.h)

```
1  #ifndef DATE_H
2  #define DATE_H
3
4  class Date
5  {
6      friend class Student;
7
8  public:
9
10     Date(int =1, int =1, int =1402);
11     ~Date(){};
12     void setDate(int, int, int);
13     int getDay() const {return day;}
14     int getMonth() const {return month;}
15     int getYear() const {return year;}
16     void print() const;
17     static int getNumDaysofMonth(int m) {return DaysOfMonth[m];}
18
19 private:
20     int day;
21     int month;
22     int year;
23     static int DaysOfMonth[];
24     bool kabise() {return false;} // if the year is a leap year, return true; otherwise, return false
25     void setDay(int);
26     void setMonth(int);
27     void setYear(int);
28
29 };
30 #endif // DATE_H
31
```



مثال ۷: (فایل Date.cpp)

```
1  #include <iostream>
2  #include "Date.h"
3
4  using namespace std;
5
6  int Date::DaysOfMonth[]={0,31,31,31,31,31,31,30,30,30,30,30,29};
7
8  Date::Date(int d,int m,int y)
9  {
10     setDate(d, m, y);
11 }
12 void Date::setYear(int y)
13 {
14     if(y>0)
15         year = y;
16     else
17     {
18         cout<<"year is invalid! set year=1402."<<endl;
19         year = 1402;
20     }
21 }
22
23 void Date::setMonth(int m)
24 {
25     if(m>=1 && m<=12)
26         month = m;
27     else
28     {
29         cout<<"month is invalid! Set month =1"<<endl;
30         month = 1;
31     }
32 }
33 }
```



مثال ۷: (ادامه فایل Date.cpp)

```
34 void Date::setDay(int d)
35 {
36     if(d >= 1 && d <= DaysOfMonth[month])
37         day = d;
38     else
39         if (d == 30 && month ==12 && kabise())
40             day = d;
41         else
42             {
43                 cout<<"day is invalid! set day = 1."<<endl;
44                 day = 1;
45             }
46 }
47 void Date::setDate(int d, int m, int y)
48 {
49     setYear(y);
50     setMonth(m);
51     setDay(d);
52 }
53
54 void Date::print() const
55 {
56     cout<<year<<"/"<<month<<"/"<<day;
57 }
58
59
```



مثال ۷: (فایل Student.h)

```
1  #ifndef STUDENT_H
2  #define STUDENT_H
3
4  #include <string>
5  #include "Date.h"
6
7  using namespace std;
8
9  class Student
10 {
11 public:
12     Student(string &, Date &d);
13     Student() {}
14     ~Student() {}
15     void setName(string &);
16     void setBirthDate(const Date &);
17     string getName() const;
18     Date getBirthDate() const;
19     bool IsBirthDay(const Date&);
20 private:
21
22     string Name;
23     Date BirthDate;
24
25 };
26
27 #endif // STUDENT_H
28
```



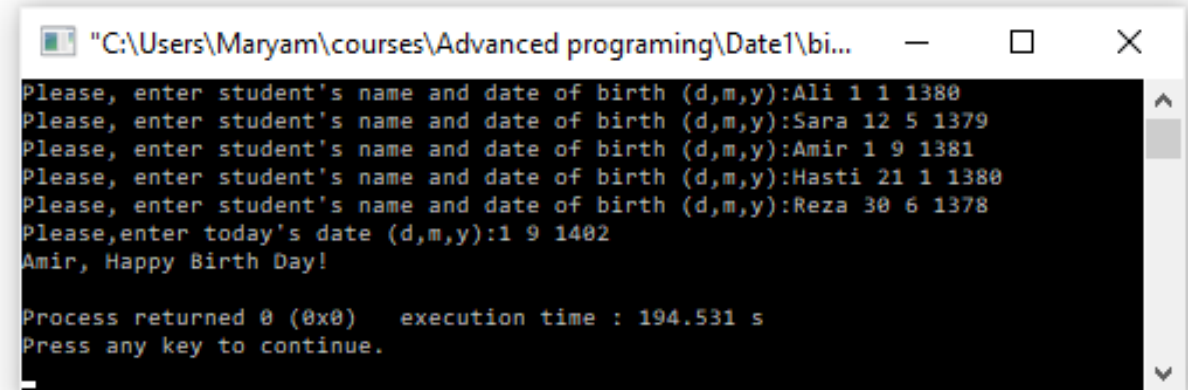
مثال ۷: (فایل Student.cpp)

```
1  #include "Student.h"
2
3  Student::Student(string &n, Date &d): BirthDate(d), Name(n)
4  {
5  }
6
7  void Student::setName(string &n)
8  {
9      Name = n;
10 }
11
12 void Student::setBirthDate(const Date &d)
13 {
14     BirthDate = d;
15 }
16
17 string Student::getName() const
18 {
19     return Name;
20 }
21
22 Date Student::getBirthDate() const
23 {
24     return BirthDate;
25 }
26
27 bool Student::IsBirthDay(const Date& d)
28 {
29     if(BirthDate.day == d.day && BirthDate.month == d.month)
30         return true;
31     else
32         return false;
33 }
34 }
35
```



مثال ۷: (فایل main.cpp)

```
1  #include <iostream>
2  #include "Date.h"
3  #include "Student.h"
4
5  using namespace std;
6
7  int main()
8  {
9      const int SIZE = 5;
10     Date today, date;
11     const Date Nowruz(1,1,1402);
12     Student s[SIZE];
13     string name;
14     int d, m, y;
15
16     for(int i=0; i< SIZE; i++)
17     {
18         cout << "Please, enter student's name and date of birth (d,m,y)
19         cin >> name >> d >> m >> y;
20         date.setDate(d,m,y);
21         s[i].setName(name);
22         s[i].setBirthDate(date);
23     }
24
25     cout << "Please,enter today's date (d,m,y):";
26     cin >> d >> m >> y;
27     today.setDate(d,m,y);
28
29     for(int i=0;i<SIZE;i++)
30     {
31         if(s[i].IsBirthDay(today))
32             cout << s[i].getName() << ", Happy Birth Day! " << endl;
33     }
34
35     return 0;
36 }
```



```
"C:\Users\Maryam\courses\Advanced programing\Date1\bi...
Please, enter student's name and date of birth (d,m,y):Ali 1 1 1380
Please, enter student's name and date of birth (d,m,y):Sara 12 5 1379
Please, enter student's name and date of birth (d,m,y):Amir 1 9 1381
Please, enter student's name and date of birth (d,m,y):Hasti 21 1 1380
Please, enter student's name and date of birth (d,m,y):Reza 30 6 1378
Please,enter today's date (d,m,y):1 9 1402
Amir, Happy Birth Day!

Process returned 0 (0x0)   execution time : 194.531 s
Press any key to continue.
```

